



University of Information Technology & Sciences

Assignment - 1

Department : CSE
Program : CSE (Dual)
Course Title : Database Management System Lab
Course code : CSE0612216
Section : 3A2
Batch : 56
Semester : Autumn 2025

Submitted to :

Pabon Shaha

Designation: Lecturer

Department: CSE

Submitted By:

Name: Badhon Kumar Roy

ID : 04324205101042

Submission Date : 07 / 09 / 2025

1. Create the below Sales table.

sale_id	product_id	quantity_sold	sale_date	total_price
1	101	5	2024-01-01	2500.00
2	102	3	2024-01-02	900.00
3	103	2	2024-01-02	60.00
4	104	4	2024-01-03	80.00
5	105	6	2024-01-03	90.00

Sales Table

SQL –

```
1 CREATE TABLE sales(  
2     sale_id INT PRIMARY KEY,  
3     product_id INT,  
4     quantity_sold INT NOT NULL,  
5     sale_date DATE NOT NULL,  
6     total_price DECIMAL(10,2),  
7     FOREIGN KEY (product_id) REFERENCES product(product_id)  
8 );|
```

2. Also create the below product table.

product_id	product_name	category	unit_price
101	Laptop	Electronics	500.00
102	Smartphone	Electronics	300.00
103	Headphones	Electronics	30.00
104	Keyboard	Electronics	20.00
105	Mouse	Electronics	15.00

SQL-

```
1 CREATE TABLE product(  
2     product_id int PRIMARY KEY,  
3     product_name VARCHAR(50) NOT NULL,  
4     category VARCHAR(50) NOT NULL,  
5     unit_price DECIMAL(10,2)  
6 );|
```

3. Retrieve all columns from the Sales table.

SQL-

```
1 SELECT * FROM sales
```

Output-

← T →						sale_id	product_id	quantity_sold	sale_date	total_price		
☐		Edit		Copy		Delete	1	101	5	2024-01-01	2500.00	
☐		Edit		Copy		Delete	2	102	3	2024-01-02	900.00	
☐		Edit		Copy		Delete	3	103	2	2024-01-02	60.00	
☐		Edit		Copy		Delete	4	104	4	2024-01-03	80.00	
☐		Edit		Copy		Delete	5	105	6	2024-01-03	90.00	
	☐	Check all	With selected:			Edit		Copy		Delete		Export

4. Retrieve the product_name and unit_price from the Products table.

SQL-

```
1 SELECT product_name, unit_price FROM product
```

Output-

				product_name	unit_price		
☐	 Edit	 Copy	 Delete	Laptop	500.00		
☐	 Edit	 Copy	 Delete	Smartphone	300.00		
☐	 Edit	 Copy	 Delete	Headphones	30.00		
☐	 Edit	 Copy	 Delete	Keyboard	20.00		
☐	 Edit	 Copy	 Delete	Mouse	15.00		
	☐ Check all	With selected:		 Edit	 Copy	 Delete	 Export

5. Retrieve the sale_id and sale_date from the Sales table.

SQL-

```
1 SELECT sale_id, sale_date FROM sales
```

Output-

			sale_id	sale_date		
☐	 Edit	 Copy	 Delete	1 2024-01-01		
☐	 Edit	 Copy	 Delete	2 2024-01-02		
☐	 Edit	 Copy	 Delete	3 2024-01-02		
☐	 Edit	 Copy	 Delete	4 2024-01-03		
☐	 Edit	 Copy	 Delete	5 2024-01-03		
	☐ Check all	With selected:	 Edit	 Copy	 Delete	 Export

6. Filter the Sales table to show only sales with a total_price greater than \$100.

SQL-

```
1 SELECT * FROM sales WHERE total_price > 100;
```

Output-

		sale_id	product_id	quantity_sold	sale_date	total_price
☐	 Edit  Copy  Delete	1	101	5	2024-01-01	2500.00
☐	 Edit  Copy  Delete	2	102	3	2024-01-02	900.00
	☐ Check all With selected:  Edit  Copy  Delete  Export					

7. Filter the Products table to show only products in the 'Electronics' category.

SQL-

```
1 SELECT * FROM product WHERE category = 'Electronics';
```

Output-

				product_id	product_name	category	unit_price
☐	 Edit	 Copy	 Delete	101	Laptop	Electronics	500.00
☐	 Edit	 Copy	 Delete	102	Smartphone	Electronics	300.00
☐	 Edit	 Copy	 Delete	103	Headphones	Electronics	30.00
☐	 Edit	 Copy	 Delete	104	Keyboard	Electronics	20.00
☐	 Edit	 Copy	 Delete	105	Mouse	Electronics	15.00
	☐ Check all	With selected:	 Edit	 Copy	 Delete	 Export	

8. Retrieve the sale_id and total_price from the Sales table for sales made on January 3, 2024.

SQL-

```
1 SELECT sale_id, total_price FROM sales WHERE sale_date = '2024-01-03';
```

Output-

				sale_id	total_price			
☐		Edit		Copy		Delete	4	80.00
☐		Edit		Copy		Delete	5	90.00

☐ Check all With selected:  Edit  Copy  Delete  Export

9. Retrieve the product_id and product_name from the Products table for products with a unit_price greater than \$100.

SQL-

```
1 SELECT product_id, product_name FROM product WHERE unit_price > 100;
```

Output-

		product_id	product_name
☐	Edit	Copy	Delete
		101	Laptop
☐	Edit	Copy	Delete
		102	Smartphone
	☐ Check all	With selected:	Edit Copy Delete Export

10. Calculate the total revenue generated from all sales in the Sales table.

SQL-

```
1 SELECT SUM(total_price) FROM sales;
```

Output-

SUM(total_price)
3630.00

11. Calculate the average unit_price of products in the Products table.

SQL-

```
1 SELECT AVG(unit_price) FROM product;
```

Output-

AVG(unit_price)
173.000000

12. Calculate the total quantity_sold from the Sales table.

SQL-

```
1 SELECT SUM(quantity_sold) FROM sales;
```

Output-

SUM(quantity_sold)
20

13. Count Sales Per Day from the Sales table.

SQL-

```
1 SELECT sale_date, COUNT(sale_id) AS total_sales
2 FROM Sales
3 GROUP BY sale_date
4 ORDER BY sale_date;
5
```

Output-

sale_date	total_sales
2024-01-01	1
2024-01-02	2
2024-01-03	2

14. Retrieve product_name and unit_price from the Products table with the Highest Unit Price.

SQL-

```
1 SELECT product_name, unit_price
2 FROM product
3 WHERE unit_price = (SELECT MAX(unit_price) FROM product);
```

Output-

	product_name	unit_price
☐  Edit  Copy  Delete	Laptop	500.00
 ☐ Check all With selected:  Edit  Copy  Delete  Export		

15. Retrieve the sale_id, product_id, and total_price from the Sales table for sales with a quantity_sold greater than 4.

SQL-

```
1 SELECT sale_id, product_id, total_price
2 FROM sales WHERE quantity_sold > 4;
```

Output-

	sale_id	product_id	total_price
☐  Edit  Copy  Delete	1	101	2500.00
☐  Edit  Copy  Delete	5	105	90.00
 ☐ Check all With selected:  Edit  Copy  Delete  Export			

16. Retrieve the product_name and unit_price from the Products table, ordering the results by unit_price in descending order.

SQL-

```
1 SELECT product_name, unit_price
2 FROM product
3 ORDER BY unit_price DESC;
```

Output-

				product_name	unit_price
☐	Edit	Copy	Delete	Laptop	500.00
☐	Edit	Copy	Delete	Smartphone	300.00
☐	Edit	Copy	Delete	Headphones	30.00
☐	Edit	Copy	Delete	Keyboard	20.00
☐	Edit	Copy	Delete	Mouse	15.00

☐ Check all With selected: Edit Copy Delete Export

17. Retrieve the total_price of all sales, rounding the values to two decimal places.

SQL-

```
1 SELECT ROUND(total_price, 2) AS rounded_total_price
2 FROM sales;
```

Output-

rounded_total_price
2500.00
900.00
60.00
80.00
90.00

18. Calculate the average total_price of sales in the Sales table.

SQL-

```
1 SELECT AVG(total_price) AS avg_total_price
2 FROM Sales;
```

Output-

avg_total_price
726.000000

19. Retrieve the product_name and unit_price from the Products table, filtering the unit_price to show only values between \$20 and \$600.

SQL-

```
1 SELECT product_name, unit_price
2 FROM product
3 WHERE unit_price BETWEEN 20 AND 600;
```

Output-

				product_name	unit_price
☐				Laptop	500.00
☐				Smartphone	300.00
☐				Headphones	30.00
☐				Keyboard	20.00

 ☐ Check all With selected:  Edit  Copy  Delete  Export

20. Retrieve the product_name and category from the Products table, ordering the results by category in ascending order.

SQL-

```
1 SELECT product_name, category
2 FROM product
3 ORDER BY category ASC;
```

Output-

</

-----X-----